

Kimchi's Inner-Product-Argument Polynomial Commitment

A blueprint, as an ArkLib `Commitment.Scheme` instance

Formalization roadmap (Halo §3 / kimchi `ipa.rs`)

June 29, 2026

Contents

1	The ArkLib Commitment-Scheme Interface	2
1.1	The scheme	2
1.2	Correctness	3
1.3	Evaluation binding	3
2	The IPA Polynomial Commitment	5
2.1	The structured reference string	5
2.2	The Pedersen commitment	5
2.3	The challenge polynomial	6
2.4	The opening relation	6
2.5	The opening proof transcript	6
2.6	The interactive opening protocol	7
2.7	The commitment scheme instance	8
2.8	Mathlib dependency anchors	8
3	Perfect Completeness of the IPA Scheme	10
3.1	Perfect completeness	10
4	Binding / Knowledge Soundness of the IPA Scheme	12
4.1	Witness-extended emulation	12
5	The IPA Scheme over the Pasta Curves	15
5.1	CompElliptic Pasta dependency anchors	15
5.2	The Pasta instances	16
5.3	Security at the Pasta curves	16

Chapter 1

The ArkLib Commitment-Scheme Interface

This chapter records the ArkLib *functional commitment scheme* interface that the kimchi IPA construction implements. Every block below is a *dependency anchor*: it states an external ArkLib declaration (namespace `Commitment`) in the project’s notation and marks it

✓ provided upstream (Mathlib/ArkLib)

, so the dependency graph treats it as supplied. None of these are proof obligations of this project — the

⊢ Lean:

target is the citation. The IPA chapters (`IPA_Basic`, `IPA_Correctness`, `IPA_Binding`) depend on these labels.

Throughout, fix an oracle specification $\mathcal{O}_{\text{spec}}$ and a data type `Data` equipped with an `OracleInterface`, i.e. a query type `Query`, a response family `Response`, and an answering map $\text{answer} : \text{Data} \rightarrow (q : \text{Query}) \rightarrow \text{Response } q$. A scheme is parameterized by types `Commitment`, `Decommitment`, `ComKey`, `VerifKey`, and a protocol specification `pSpec` for the opening proof.

1.1 The scheme

Definition 1.1 (Key generation). ⊢ Lean: `Commitment.KeyGen`

✓ provided upstream (Mathlib/ArkLib)

Provided by ArkLib. Key generation is an oracle computation $\text{KeyGen} : \text{OracleComp } \mathcal{O}_{\text{spec}} (\text{ComKey} \times \text{VerifKey})$ that samples a committer key and a verifier key.

Definition 1.2 (Commit). ⊢ Lean: `Commitment.Commit`

✓ provided upstream (Mathlib/ArkLib)

Provided by ArkLib. The commit algorithm is a map $\text{Commit} : \text{ComKey} \rightarrow \text{Data} \rightarrow \text{OracleComp } \mathcal{O}_{\text{spec}} (\text{Commitment} \times \text{Decommitment})$, taking a committer key and the data, and producing a commitment together with a decommitment value (auxiliary information such as blinding randomness needed to open later; $\text{Decommitment} = \text{Unit}$ for deterministic schemes).

Definition 1.3 (Opening proof). ⊢ Lean: `Commitment.Opening`

✓ provided upstream (Mathlib/ArkLib)

Provided by ArkLib. The opening component supplies, for each key pair (ck, vk) , an interactive proof relative to $\mathcal{O}_{\text{spec}}$ over the protocol specification pSpec for the relation on statements (cm, q, y) with $q : \text{Query}$, $y : \text{Response } q$ and witnesses (d, dc) :

$$\text{Opening} : (cm, q, y), (d, dc) \mapsto (\text{Commit } ck \ d \rightsquigarrow (cm, dc)) \wedge \text{answer } d \ q = y,$$

i.e. that d commits to cm with decommitment dc and that the oracle on d answers q with y .

Definition 1.4 (Commitment scheme). $\vdash \text{Lean: Commitment.Scheme}$

✓ provided upstream (Mathlib/ArkLib)

Provided by ArkLib. A commitment scheme relative to $\mathcal{O}_{\text{spec}}$ for data Data with an `OracleInterface` is the bundle of a key generator (Definition 1.1), a commit algorithm (Definition 1.2), and an opening proof (Definition 1.3) over a fixed protocol specification pSpec .

1.2 Correctness

Definition 1.5 (Correctness). $\vdash \text{Lean: Commitment.correctness}$

✓ provided upstream (Mathlib/ArkLib)

Provided by ArkLib. A scheme satisfies **correctness** with error $\varepsilon_c \in \mathbb{R}_{\geq 0}$ if for every $\text{data} : \text{Data}$ and every $\text{query} : \text{Query}$, running key generation, the honest commit on data , and then the honest opening proof on the statement $(cm, \text{query}, \text{answer } \text{data } \text{query})$ makes the verifier accept (with the prover's output statement equal to the verifier's) with probability at least $1 - \varepsilon_c$.

Definition 1.6 (Perfect correctness). $\vdash \text{Lean: Commitment.perfectCorrectness}$

✓ provided upstream (Mathlib/ArkLib)

Provided by ArkLib. A scheme satisfies *perfect correctness* if it satisfies **correctness** (Definition 1.5) with error 0.

1.3 Evaluation binding

Definition 1.7 (Binding adversary). $\vdash \text{Lean: Commitment.BindingAdversary}$

✓ provided upstream (Mathlib/ArkLib)

Provided by ArkLib. A binding adversary (for an auxiliary-state type AuxState) consists of: a *claim* step, an oracle computation $\text{ComKey} \rightarrow \text{OracleComp } \mathcal{O}_{\text{spec}} (\text{Commitment} \times (q : \text{Query}) \times \text{Response } q \times \text{Response } q \times \text{AuxState} \times \text{AuxState})$ that outputs a commitment cm , a query q , two purported responses, and two auxiliary states; and a malicious *prover* for the opening protocol that consumes such an auxiliary state.

Definition 1.8 (Binding condition). $\vdash \text{Lean: Commitment.bindingCondition}$

✓ provided upstream (Mathlib/ArkLib)

Provided by ArkLib. The win condition for the binding game, on a tuple $(q, \text{resp}_1, \text{resp}_2, \text{accept}_1, \text{accept}_2)$, holds iff

$$\text{resp}_1 \neq \text{resp}_2 \wedge \text{accept}_1 \wedge \text{accept}_2,$$

i.e. the two responses to the same query differ yet both openings are accepted.

Definition 1.9 (Binding experiment). $\vdash$ Lean: `Commitment.bindingExperiment`

✓ provided upstream (Mathlib/ArkLib)

Provided by ArkLib. The binding experiment is the probability that a fixed adversary (Definition 1.7) wins the evaluation-binding game, i.e. the probability that the play of claim-then-two-openings satisfies the binding condition (Definition 1.8): $\Pr[\text{bindingCondition} \mid \text{bindingGame}] \in \overline{\mathbb{R}}_{\geq 0}$.

Definition 1.10 (Evaluation binding). $\vdash$ Lean: `Commitment.binding`

✓ provided upstream (Mathlib/ArkLib)

Provided by ArkLib. A scheme satisfies (evaluation) binding with error $\varepsilon_b \in \mathbb{R}_{\geq 0}$ if for every auxiliary-state type and every adversary the binding experiment (Definition 1.9) is at most ε_b . Informally, it is infeasible to open a commitment to two different responses for the same query.

Chapter 2

The IPA Polynomial Commitment

This chapter constructs kimchi’s inner-product-argument (IPA) univariate polynomial commitment as an instance of ArkLib’s `Commitment.Scheme`. The scheme is generic over an additive commitment group $\mathbb{G}$ of prime order with scalar field $\mathbb{F}_r$: a degree- $(d - 1)$ polynomial over $\mathbb{F}_r$ is committed by a Pedersen vector commitment to its coefficient vector, and the prover convinces a verifier of a claimed evaluation through a logarithmic-round folding argument wrapped in a zero-knowledge Schnorr step. The mathematical statement follows Halo §3; the operational ground truth is the kimchi Rust reference (`ipa.rs`, `commitment.rs`). The security theorems (correctness, binding) and the Pasta instantiation are deferred to their own chapters.

2.1 The structured reference string

Definition 2.1 (IPA structured reference string). $\vdash$ Lean: `Kimchi.Commitment.IPA.SRS`

Source: Halo, Section 3 (Setup); kimchi `ipa.rs struct SRS`. Fix a degree bound $d - 1$ and assume $d = 2^k$. The structured reference string produced by `Setup(1 $^\lambda$ , d)` consists of a vector of generators $g = (g_0, \dots, g_{d-1}) \in \mathbb{G}^d$, used to commit to a polynomial in coefficient form, together with a single blinding base $H \in \mathbb{G}$. All of $g_0, \dots, g_{d-1}, H$ are sampled uniformly at random and independently; no trapdoor or hidden structure relates them, so the only constraint the SRS imposes is that no nontrivial discrete-log relation among its elements is known. In the Rust reference these are exactly the fields `g` (the commitment generators) and `h` (the blinding base) of `struct SRS`.

2.2 The Pedersen commitment

Definition 2.2 (IPA Pedersen commitment). $\vdash$ Lean: `Kimchi.Commitment.IPA.commit`

Source: Halo, Section 3 (Commit); kimchi `ipa.rs commit_non_hiding/mask`. Let $p(X) \in \mathbb{F}_r[X]$ have degree at most $d - 1$, and write $a = (a_0, \dots, a_{d-1})$ for its coefficient vector, so that a_i is the coefficient of the degree- i term. For a blinding factor $r \in \mathbb{F}_r$ the commitment relative to the SRS $\sigma = (g, H)$ of Definition 2.1 is the Pedersen vector commitment

$$\text{Commit}(\sigma, p(X); r) = \langle a, g \rangle + [r]H = \sum_{i=0}^{d-1} [a_i]g_i + [r]H,$$

where $\langle a, g \rangle = \sum_i [a_i]g_i$ is the multiscalar multiplication of the coefficients against the generators and $[r]H$ is the scalar action of r on the blinding base. The non-hiding part $\langle a, g \rangle$ (`commit_non_hiding`) is turned into the hiding commitment by adding $[r]H$ for a freshly sampled r (`mask`), which records

r as the decommitment. The commitment is perfectly hiding and additively homomorphic: for all $a, b, r, s \in \mathbb{F}_r$ and $p, q \in \mathbb{F}_r[X]$,

$$[a] \text{Commit}(\sigma, p(X); r) + [b] \text{Commit}(\sigma, q(X); s) = \text{Commit}(\sigma, a \cdot p(X) + b \cdot q(X); ar + bs).$$

2.3 The challenge polynomial

Definition 2.3 (challenge polynomial). $\vdash$ Lean: `Kimchi.Commitment.IPA.bPoly`

Source: *Halo*, Section 3.2 eq. (3) and Section 3.1 eq. (1); `kimchi commitment.rs::b_poly`. Given the k folding challenges $u_1, \dots, u_k \in \mathbb{F}_r$, Halo’s challenge polynomial is

$$g(X, u_1, \dots, u_k) = \prod_{i=1}^k (u_i + u_i^{-1} X^{2^{i-1}}).$$

Its value at the evaluation point x equals the inner product $\langle s, (1, x, x^2, \dots, x^{d-1}) \rangle$, where s is the length- d vector of eq. (1) whose entry at index $\sum_j b_j 2^j$ is the product $\prod_{j: b_j=1} u_{k-j}^{\pm 1}$ of (inverse) challenges dictated by the binary expansion of the index; concretely s records, for the first half of bases scaled by inverse challenges and the second half by the plain challenges, the binary-counting product structure of the fold. kimchi deploys the reindexed and normalized variant

$$b(X) = \prod_{i=0}^{k-1} (1 + u_{k-i} X^{2^i}),$$

computed by `commitment.rs::b_poly` (which forms the squarings x^{2^i} and multiplies the factors $1 + u_{k-i} x^{2^i}$); this form satisfies $b(x) = \langle s, (1, x, \dots, x^{d-1}) \rangle$ for the same vector s . The folded evaluation point produced after all rounds of the protocol of Definition 2.6 equals $b(x)$, so the verifier can recompute it in time $O(k) = O(\log d)$ rather than by a linear scan.

2.4 The opening relation

Definition 2.4 (IPA opening relation). $\vdash$ Lean: `Kimchi.Commitment.IPA.openingRelation`

Source: *Halo*, Section 3 (the opening relation). The opening relation pairs a public statement (P, x, v) — a commitment $P \in \mathbb{G}$, an evaluation point $x \in \mathbb{F}_r$, and a claimed value $v \in \mathbb{F}_r$ — with a witness (a, r) , the coefficient vector $a \in \mathbb{F}_r^d$ and the blinding factor $r \in \mathbb{F}_r$. The pair is in the relation when

$$P = \langle a, g \rangle + [r]H \quad \text{and} \quad v = \langle a, (1, x, x^2, \dots, x^{d-1}) \rangle.$$

The first conjunct asserts that P is the Pedersen commitment of Definition 2.2 to the polynomial with coefficients a under blinding r ; the second asserts that this polynomial evaluates to v at x , i.e. $v = p(x)$ where $p(X) = \sum_i a_i X^i$. Membership thus certifies both that the committed polynomial has degree at most $d - 1$ and that it opens to v at x .

2.5 The opening proof transcript

Definition 2.5 (IPA opening proof). $\vdash$ Lean: `Kimchi.Commitment.IPA.OpeningProof`

Source: `kimchi ipa.rs struct OpeningProof` (and *Halo*, Section 3.1, the Schnorr responses). An opening proof for a statement of Definition 2.4 is the transcript emitted by the prover. It consists of:

- the $k = \log_2 d$ rounds of cross-term commitments $(L_j, R_j) \in \mathbb{G} \times \mathbb{G}$, one pair per folding round (the field `1r`);
- the final folded scalar $a \in \mathbb{F}_r$, the length-one residue of the coefficient vector after all k folds (denoted a_0 below);
- the final folded commitment base $sg = g_0 \in \mathbb{G}$, the length-one residue of the generator vector after folding (the field `sg`);
- the Schnorr blinding commitment $\delta \in \mathbb{G}$ (Halo's R , the field `delta`); and
- the Schnorr responses $z_1 = ac + d$ and $z_2 = cr' + s$ in $\mathbb{F}_r$ (the fields `z1`, `z2`), where c is the verifier's Schnorr challenge, d, s are the prover's fresh blinders, and r' is the synthetic blinding factor accumulated across the fold.

The folded base sg doubles as the value the verifier checks against $\langle s, g \rangle$ for the vector s of Definition 2.3.

2.6 The interactive opening protocol

Definition 2.6 (IPA opening protocol). $\vdash$ `Lean: Kimchi.Commitment.IPA.openingProof`

Source: Halo, Section 3.1 and eq. (2); kimchi `ipa.rs open/verify`. The protocol takes the commitment P , the point x , and the claimed value v as common inputs, and is an argument of knowledge for the relation of Definition 2.4 (instantiating the ArkLib opening interface of Definition 1.3). It proceeds as follows.

Setup move. The verifier sends a random group element $U \in \mathbb{G}$, and both parties form

$$P' = P + [v]U.$$

The prover initializes $g' := g$, $a' := a$, and the fixed evaluation vector $b' := (1, x, x^2, \dots, x^{d-1})$.

Folding rounds. For $j = k, k-1, \dots, 1$, split each of g', a', b' into its lower and upper halves. The prover samples blinders $l_j, r_j \in \mathbb{F}_r$ and sends the cross-term commitments

$$L_j = \langle a'_{\text{lo}}, g'_{\text{hi}} \rangle + [l_j]H + [\langle a'_{\text{lo}}, b'_{\text{hi}} \rangle]U, \quad R_j = \langle a'_{\text{hi}}, g'_{\text{lo}} \rangle + [r_j]H + [\langle a'_{\text{hi}}, b'_{\text{lo}} \rangle]U.$$

The verifier replies with a challenge $u_j \in \mathbb{F}_r$, and both parties fold

$$a' \leftarrow a'_{\text{hi}} u_j^{-1} + a'_{\text{lo}} u_j, \quad b' \leftarrow b'_{\text{lo}} u_j^{-1} + b'_{\text{hi}} u_j, \quad g' \leftarrow g'_{\text{lo}} u_j^{-1} + g'_{\text{hi}} u_j.$$

After k rounds each of g', a', b' has length one; write $a = a'_0$, $g_0 = g'_0$, and note $b'_0 = b(x)$ for the challenge polynomial b of Definition 2.3 reconstructed from the u_j .

Recombination. The verifier forms

$$Q = \sum_{j=1}^k [u_j^2] L_j + P' + \sum_{j=1}^k [u_j^{-2}] R_j,$$

and writes $G = \langle s, g \rangle = g_0$ and $b = \langle s, (1, x, \dots, x^{d-1}) \rangle = b(x)$ for the vector s of Definition 2.3. The synthetic blinder is $r' = \sum_j (l_j u_j^2) + r + \sum_j (r_j u_j^{-2})$, and the reduced claim is knowledge of a, r' with

$$Q = [a]G + [r']H + [ab]U = [a](G + [b]U) + [r']H.$$

Zero-knowledge step. The prover samples $d, s \in \mathbb{F}_r$ and sends the blinding commitment

$$\delta = [d] (G + [b]U) + [s]H$$

(Halo’s R). The verifier replies with a Schnorr challenge $c \in \mathbb{F}_r$, and the prover answers with

$$z_1 = ac + d, \quad z_2 = cr' + s.$$

The verifier accepts iff

$$[c] Q + \delta = [z_1] (G + [b]U) + [z_2] H.$$

The transcript $(\{(L_j, R_j)\}_j, a, g_0, \delta, z_1, z_2)$ is exactly the opening proof of Definition 2.5.

2.7 The commitment scheme instance

Definition 2.7 (IPA commitment scheme). $\vdash$ Lean: `Kimchi.Commitment.IPA.ipa`

Source: `ArkLib KZG_Basic.lean (def kzg)`, the worked `Commitment.Scheme` instance whose shape this mirrors. The IPA scheme is the `Commitment.Scheme` instance of Definition 1.4 obtained by the following choices of the abstract data slots:

- **Data** is a polynomial of degree $< d$, presented as its coefficient vector in $\mathbb{F}_r^d$, equipped with the evaluation `OracleInterface` whose query at a point $z \in \mathbb{F}_r$ returns the evaluation $p(z) = \sum_i a_i z^i$;
- **Commitment** $= \mathbb{G}$, a single group element;
- **Decommitment** $= \mathbb{F}_r$, the blinding factor r (in contrast to KZG, where it is `Unit`, the IPA scheme is randomized);
- **ComKey** $=$ **VerifKey** $=$ **SRS**, the structured reference string of Definition 2.1;
- **KeyGen** samples the SRS (the random generators g and blinding base H);
- **Commit** is the Pedersen commitment of Definition 2.2, returning $(\langle a, g \rangle + [r]H, r)$; and
- **Opening** is the interactive opening protocol of Definition 2.6.

The same committer and verifier keys hold the full SRS, since both parties need the generator vector to run the fold.

2.8 Mathlib dependency anchors

Definition 2.8 (univariate polynomial evaluation). $\vdash$ Lean: `Polynomial.eval`

✓ provided upstream (Mathlib/ArkLib)

Provided by Mathlib. For $p \in \mathbb{F}_r[X]$ and $z \in \mathbb{F}_r$, `Polynomial.eval z p` is the value $p(z) = \sum_i a_i z^i$ of p at z . This is the evaluation underlying both the second conjunct of the opening relation (Definition 2.4) and the `OracleInterface` query of the scheme (Definition 2.7).

Definition 2.9 (finite sum). $\vdash$ Lean: `Finset.sum`

✓ provided upstream (Mathlib/ArkLib)

Provided by Mathlib. For a finite index set I and a family $f : I \rightarrow M$ valued in a commutative monoid, `Finset.sum` forms $\sum_{i \in I} f(i)$. The multiscalar multiplication $\langle a, g \rangle = \sum_{i=0}^{d-1} [a_i]g_i$ of Definition 2.2 is such a finite sum over $I = \{0, \dots, d-1\}$.

Definition 2.10 (scalar action). $\vdash$ Lean: `SMul.smul`

✓ provided upstream (Mathlib/ArkLib)

Provided by Mathlib. For a scalar $a \in \mathbb{F}_r$ and a group element $P \in \mathbb{G}$, `SMul.smul a P` is the scalar action $[a]P$ of the field $\mathbb{F}_r$ on the additive group $\mathbb{G}$. Every $[a_i]g_i$, $[r]H$, $[v]U$, and $[c]Q$ appearing in Definition 2.2 and Definition 2.6 is an instance of this action.

Chapter 3

Perfect Completeness of the IPA Scheme

This chapter states and sketches the first of the two headline security properties of the IPA polynomial commitment of Definition 2.7: *perfect completeness*. Informally, an honest prover always convinces the verifier of a true evaluation: committing to a polynomial p of degree $< d$ and then running the honest opening protocol on a point x with the true value $v = p(x)$ makes the verifier accept with probability 1. This is the completeness half of Halo’s Theorem 1; binding and extractability are deferred to their own chapter.

The Lean statement follows the two-layer shape used by ArkLib’s worked KZG development. There, an *algebraic* correctness lemma (`KZG.correctness`), asserting that the concrete commit/open/verify operations satisfy the verification equation, is *lifted* to the interface-level predicate `Commitment.perfectCorrectness` (`KZG.CommitmentScheme.correctness`) by unfolding the oracle-computation plumbing of `KeyGen`, `Commit`, and the opening reduction and discharging the probability-one obligation. The IPA correctness theorem below plays the role of that interface-level statement: its proof reduces, after the same unfolding, to the algebraic accept-equation identity sketched here for the honest transcript of Definition 2.6.

3.1 Perfect completeness

Theorem 3.1 (Perfect completeness of IPA). $\vdash \text{Lean: Kimchi.Commitment.IPA.Correctness.ipa_correctness}$

Source: Halo, Section 3, Theorem 1 (perfect completeness). The IPA commitment scheme `ipa` of Definition 2.7 satisfies perfect correctness in the sense of Definition 1.6: for every polynomial $p \in \mathbb{F}_r[X]$ of degree $< d$, presented by its coefficient vector $a \in \mathbb{F}_r^d$, and every evaluation point $x \in \mathbb{F}_r$, the experiment that

1. *runs key generation to obtain the SRS $\sigma = (g, H)$,*
2. *runs the honest `Commit` of Definition 2.2 on a , yielding the commitment $P = \langle a, g \rangle + [r]H$ with decommitment r , and*
3. *runs the honest opening protocol of Definition 2.6 on the statement (P, x, v) with $v = p(x)$,*

makes the verifier accept with probability 1; equivalently, `ipa` satisfies correctness with error $\varepsilon_c = 0$.

Proof. Source: Halo, Section 3.1 and eq. (2). The proof mirrors the perfect-completeness half of Halo’s Theorem 1: it tracks the honest prover’s values through the protocol of Definition 2.6

and verifies the final accept equation by direct substitution. Since the honest `Commit` makes the statement (P, x, v) a member of the opening relation of Definition 2.4 (i.e. $P = \langle a, g \rangle + [r]H$ and $v = p(x) = \langle a, (1, x, \dots, x^{d-1}) \rangle$), every step below is an identity in $\mathbb{G}$ and $\mathbb{F}_r$, so the accepting event has probability 1. At the interface level this algebraic identity is what the `perfectCorrectness` unfolding reduces to, exactly as `ArkLib's KZG.CommitmentScheme.correctness` reduces to its algebraic `KZG.correctness` lemma.

(i) *Collapse of the fold.* After the setup move both parties hold $P' = P + [v]U$, and the prover initializes $g' := g$, $a' := a$, $b' := (1, x, x^2, \dots, x^{d-1})$. Each of the $k = \log_2 d$ honest folding rounds replaces a length- $2m$ triple (g', a', b') by the length- m triple

$$a' \leftarrow a'_{\text{hi}} u_j^{-1} + a'_{\text{lo}} u_j, \quad b' \leftarrow b'_{\text{lo}} u_j^{-1} + b'_{\text{hi}} u_j, \quad g' \leftarrow g'_{\text{lo}} u_j^{-1} + g'_{\text{hi}} u_j.$$

After all k rounds the three vectors have length one. Write a_0 for the final folded scalar a'_0 , $g_0 := g'_0$, and $b_0 := b'_0$. Unwinding the recursion expresses g_0 and b_0 as the inner products of the length- d coefficient vector s of Definition 2.3 (eq. (1)) against g and against the powers of x :

$$g_0 = \langle s, g \rangle =: G, \quad b_0 = \langle s, (1, x, x^2, \dots, x^{d-1}) \rangle = b(x) =: b,$$

the second equality being the defining property of the challenge polynomial b of Definition 2.3.

(ii) *Telescoping to Q (eq. (2)).* The verifier forms

$$Q = \sum_{j=1}^k [u_j^2] L_j + P' + \sum_{j=1}^k [u_j^{-2}] R_j.$$

Substituting the honest cross-term definitions $L_j = \langle a'_{\text{lo}}, g'_{\text{hi}} \rangle + [l_j]H + [\langle a'_{\text{lo}}, b'_{\text{hi}} \rangle]U$ and $R_j = \langle a'_{\text{hi}}, g'_{\text{lo}} \rangle + [r_j]H + [\langle a'_{\text{hi}}, b'_{\text{lo}} \rangle]U$ of round j , the cross-terms telescope against P' : each round's contribution combines with the running $\langle a', g' \rangle$ - and $\langle a', b' \rangle$ -terms so that all intermediate pieces cancel, leaving the length-one residue. With the synthetic blinding factor $r' = \sum_{j=1}^k (l_j u_j^2) + r + \sum_{j=1}^k (r_j u_j^{-2})$ one obtains Halo's eq. (2):

$$Q = [a_0]G + [r']H + [a_0b]U = [a_0](G + [b]U) + [r']H.$$

(iii) *The Schnorr check.* The prover samples $d, s \in \mathbb{F}_r$, sends $\delta = [d](G + [b]U) + [s]H$ (Halo's R), receives the Schnorr challenge $c \in \mathbb{F}_r$, and answers with the honest responses $z_1 = a_0c + d$ and $z_2 = cr' + s$. Substituting these and the expressions for Q and δ into the right-hand side of the verifier's check and using bilinearity of the scalar action (Definition 2.10),

$$\begin{aligned} [z_1](G + [b]U) + [z_2]H &= [a_0c + d](G + [b]U) + [cr' + s]H \\ &= [c]([a_0](G + [b]U) + [r']H) + ([d](G + [b]U) + [s]H) \\ &= [c]Q + \delta. \end{aligned}$$

This is exactly the verifier's accept condition $[c]Q + \delta = [z_1](G + [b]U) + [z_2]H$, so the honest transcript $(\{(L_j, R_j)\}_j, a_0, g_0, \delta, z_1, z_2)$ of Definition 2.5 always passes. Hence the verifier accepts with probability 1, which is perfect completeness. $\square$

Chapter 4

Binding / Knowledge Soundness of the IPA Scheme

This chapter states the second headline security property of kimchi’s IPA polynomial commitment: *knowledge soundness*, in the form of *evaluation binding*. Where the correctness chapter says that honest openings are accepted, binding says the dual: it is computationally infeasible to make the verifier accept two *different* evaluations of the same commitment at the same point. In the Halo development this is packaged as *computational witness-extended emulation* (Halo Definition 2, Theorem 1): for any efficient malicious prover there is an emulator that, alongside a transcript indistinguishable from the real interaction, extracts a witness satisfying the opening relation (Definition 2.4) whenever the transcript accepts. We record Halo’s definition verbatim, then state the property in the ArkLib experiment shape (Definition 1.10) that the Lean development targets, and give an honest high-level sketch of the extractor argument. The full extraction proof is Halo Appendix A and is deferred to a later research phase; the sketch here is a sketch, not a proof.

4.1 Witness-extended emulation

Definition 4.1 (Computational witness-extended emulation). $\vdash \text{Lean: Kimchi.Commitment.IPA.Binding.witnessExt}$

Source: Halo, Definition 2 (computational witness-extended emulation). The opening argument (Setup, Open, VerifyOpen) has *computational witness-extended emulation* if for every deterministic polynomial-time malicious prover P^* there is an expected-polynomial-time emulator E such that for every pair of interactive adversaries (A_1, A_2) the two experiments

- sample $\sigma \leftarrow \text{Setup}(1^\lambda)$ and $(x, s) \leftarrow A_2(\sigma)$, run the real interaction $\text{tr} \leftarrow \langle P^*(\sigma, x, s), \text{VerifyOpen}(\sigma, x) \rangle$, and output $A_1(\text{tr})$; versus
- the same sampling, but with $(\text{tr}, w) \leftarrow E^O(\sigma, x)$ produced by the emulator, whose oracle $O = \langle P^*(\sigma, x, s), \text{VerifyOpen}(\sigma, x) \rangle$ permits rewinding to a chosen point and resuming with fresh verifier randomness, and output $A_1(\text{tr})$ conjoined with the soundness clause $\text{tr accepts} \Rightarrow (x, w) \in R$,

differ in acceptance probability by at most $\text{negl}(\lambda)$. Here R is the opening relation of Definition 2.4. Thus the emulator reproduces the prover’s transcript distribution while additionally extracting a witness $w = (a, r)$ that genuinely opens the commitment whenever the transcript is accepting. The non-uniform variant restricts A_1, A_2 to non-uniform polynomial time.

Witness-extended emulation (Definition 4.1) is the *intended* knowledge-soundness notion for the IPA opening argument: it is the property whose extractor recovers a genuine opening witness whenever a transcript accepts. The headline result of this chapter, Theorem 4.3, instead establishes the strictly weaker but fully well-formed notion of *evaluation binding* (Definition 1.10) — that no efficient adversary opens one commitment to two distinct evaluations at the same point. A faithful Lean statement of the full witness-extended emulation property is deferred: ArkLib’s corresponding *extractability* predicate currently has its body stubbed to $\perp$ (it has not yet been given its real definition upstream), so there is no provable target to formalize against. The knowledge-soundness formulation is therefore recorded here as the semantic goal but is not a Phase-1 target; it becomes actionable only once the real ArkLib extractability body is upstreamed.

Definition 4.2 (Discrete Log Relation Assumption). $\vdash$ Lean: `Kimchi.Commitment.IPA.Binding.dlRelationAssumption`

Source: Halo, Definition 6 (discrete log relation assumption). Let G be a group of order the scalar field $\mathbb{F}$. The *discrete log relation assumption* postulates that for every $n \geq 2$ and every efficient adversary A , if $(G_1, \dots, G_n) \leftarrow G^n$ are sampled uniformly at random, then the probability that A outputs a coefficient vector $a = (a_1, \dots, a_n) \in \mathbb{F}^n$ with some $a_i \neq 0$ and

$$\langle a, (G_1, \dots, G_n) \rangle = \sum_{i=1}^n [a_i]G_i = O$$

(the group identity) is negligible in the security parameter λ . A tuple a with $\langle a, (G_1, \dots, G_n) \rangle = O$ and at least one $a_i \neq 0$ is called a *non-trivial discrete log relation* among the generators. Writing $\varepsilon_{\text{dl}}(\lambda)$ for the maximal advantage of any efficient adversary in producing such a relation, the assumption asserts $\varepsilon_{\text{dl}}(\lambda)$ is negligible. This is a postulated hardness assumption — a foundational cryptographic hypothesis taken as given, not derived from the elliptic-curve group law — and it is the quantity the IPA binding error term is measured against.

Theorem 4.3 (Binding of the IPA scheme). $\vdash$ Lean: `Kimchi.Commitment.IPA.Binding.ipa_binding`

Source: Halo, Theorem 1 (computational witness-extended emulation of the Section 3.1 protocol). The IPA commitment scheme `ipa` of Definition 2.7 satisfies ArkLib *evaluation binding* (Definition 1.10) with negligible error: there is a negligible $\varepsilon_{\text{b}}(\lambda)$ such that for every auxiliary-state type and every binding adversary, the binding experiment (Definition 1.9) for `ipa` is at most $\varepsilon_{\text{b}}(\lambda)$. Concretely, no efficient adversary can produce a commitment P , a point x , and two responses $v_1 \neq v_2$ together with malicious openings of the protocol of Definition 2.6 that both pass `VerifyOpen`. This is the ArkLib-experiment expression of Halo’s computational witness-extended emulation (Definition 4.1): the binding error ε_{b} is exactly the emulation gap of Halo Theorem 1 specialized to the evaluation query, so an adversary winning the binding game would refute the witness-extended emulation of the opening argument.

Proof. *Source:* Halo, Section 3.1 and Definition 6; the full construction is Halo Appendix A and is deferred to the research phase. This is a high-level sketch, not a complete proof.

Suppose, for contradiction, a binding adversary (Definition 1.7) produces a commitment P , an evaluation point x , and two distinct claims $v_1 \neq v_2$, with malicious provers that drive the opening protocol (Definition 2.6) to acceptance for both (P, x, v_1) and (P, x, v_2) . Recall that the first verifier move samples a fresh group element U and both parties pass to $P' = P + [v]U$; because the prover does not learn U before committing to P , an accepting transcript pins the claimed value to the value actually committed (Halo §3.1).

Apply the emulator E of Definition 4.1 to each accepting branch. Rewinding the $k = \log_2 d$ folding rounds with fresh challenges u_j and solving the resulting linear system across rounds extracts,

for branch i , a witness (a_i, r_i) with

$$P = \langle a_i, g \rangle + [r_i]H \quad \text{and} \quad v_i = \langle a_i, (1, x, x^2, \dots, x^{d-1}) \rangle,$$

i.e. $(P, x, v_i; a_i, r_i)$ lies in the opening relation Definition 2.4. The extractor obtained this way runs in expected polynomial time once the rewinding oracle is folded into the query log.

Now $v_1 \neq v_2$ forces $(a_1, r_1) \neq (a_2, r_2)$, yet both reconstruct the *same* P . Subtracting the two Pedersen equations gives

$$\langle a_1 - a_2, g \rangle + [r_1 - r_2]H = O,$$

a non-trivial discrete-log relation among the random generators g, H (and U , once the $[v]U$ terms are accounted for), because the coefficient vector $a_1 - a_2$ is nonzero on the monomial basis distinguishing v_1 from v_2 . By the discrete log relation assumption (Halo Definition 6) such a relation occurs only with negligible probability, so both openings can succeed only with negligible probability. Hence the binding error ε_b is the sum of the emulation gap of Definition 4.1 and the discrete-log advantage, both negligible.

The faithful construction of the rewinding emulator and the exact accounting of the extraction error are carried out in Halo Appendix A and are not reproduced here; this chapter states the result and records the argument's shape. $\square$

Chapter 5

The IPA Scheme over the Pasta Curves

The preceding chapters develop the IPA polynomial commitment generically: the scheme `ipa` of Definition 2.7 and its two security theorems are stated for an arbitrary additive commitment group $\mathbb{G}$ of prime order with scalar field $\mathbb{F}_r$. This chapter instantiates that generic development at the two Pasta curves, Pallas and Vesta, taking $\mathbb{G}$ to be a Pasta curve’s group of rational points and $\mathbb{F}_r$ to be the corresponding scalar field, both supplied by `CompElliptic`. The result is a concrete `Commitment.Scheme` on each curve together with the specialized correctness and binding statements; nothing new is proved here beyond transporting the generic results across the instantiation.

The Pallas and Vesta curves form a 2-cycle: both have the short-Weierstrass shape $y^2 = x^3 + 5$, the base field of one is the scalar field of the other, and each curve’s group of points has prime order equal to the cardinality of the partner field. Concretely, the Pallas curve is defined over the Pallas base field, and its group of rational points has order q , the cardinality of the Pallas scalar field; the Vesta curve is defined over that scalar field $\mathbb{F}_p[q]$, and its group has order p , the cardinality of the Pallas base field. Each instantiation pairs a curve’s point group with the partner curve’s base field as the scalar field, exactly as the generic scheme requires.

5.1 CompElliptic Pasta dependency anchors

The following four blocks are dependency anchors for the `CompElliptic` vendor library: each states an external declaration in the project’s notation and points

┆ Lean:

at the real `CompElliptic` target, marked

✓ provided upstream (Mathlib/ArkLib)

so the dependency graph treats it as supplied. None is a proof obligation of this project — the

┆ Lean:

target is the citation.

Definition 5.1 (Pallas curve group). ┆ Lean: `CompElliptic.Curves.Pasta.Pallas.curve`

✓ provided upstream (Mathlib/ArkLib)

Provided by CompElliptic. The Pallas curve is the short-Weierstrass elliptic curve $y^2 = x^3 + 5$ (coefficients $A = 0$, $B = 5$) over the Pallas base field. Its discriminant is a unit, so the curve is elliptic, and its set of rational points forms a finite abelian group of prime order q . This point group is the commitment group $\mathbb{G}$ of the Pallas instantiation of the IPA scheme.

Definition 5.2 (Pallas scalar field). $\vdash$ Lean: `CompElliptic.Fields.Pasta.PallasScalarField`

✓ provided upstream (Mathlib/ArkLib)

Provided by CompElliptic. The Pallas scalar field is the prime field $\mathbb{Z}/q\mathbb{Z}$, where q is the (prime) order of the Pallas point group of Definition 5.1. It carries a machine-checked Pratt/Lucas primality certificate for q , and is the scalar field $\mathbb{F}_r$ over which polynomials are committed and folding challenges are drawn in the Pallas instantiation.

Definition 5.3 (Vesta curve group). $\vdash$ Lean: `CompElliptic.Curves.Pasta.Vesta.curve`

✓ provided upstream (Mathlib/ArkLib)

Provided by CompElliptic. The Vesta curve is the short-Weierstrass elliptic curve $y^2 = x^3 + 5$ (coefficients $A = 0$, $B = 5$) over the Vesta base field, which coincides with the Pallas scalar field of Definition 5.2. Its rational points form a finite abelian group of prime order p , equal to the cardinality of the Pallas base field. This point group is the commitment group $\mathbb{G}$ of the Vesta instantiation.

Definition 5.4 (Vesta scalar field). $\vdash$ Lean: `CompElliptic.Fields.Pasta.VestaScalarField`

✓ provided upstream (Mathlib/ArkLib)

Provided by CompElliptic. The Vesta scalar field is the prime field $\mathbb{Z}/p\mathbb{Z}$, where p is the order of the Vesta point group of Definition 5.3; it coincides with the Pallas base field. It is the scalar field $\mathbb{F}_r$ of the Vesta instantiation.

5.2 The Pasta instances

Definition 5.5 (IPA scheme over Pallas). $\vdash$ Lean: `Kimchi.Commitment.IPA.ipaPallas`

The IPA commitment scheme over the Pallas curve is the generic scheme `ipa` of Definition 2.7 with the commitment group $\mathbb{G}$ taken to be the Pallas point group of Definition 5.1 and the scalar field $\mathbb{F}_r$ taken to be the Pallas scalar field of Definition 5.2. All of the abstract data slots of Definition 2.7 — the structured reference string of generators and blinding base, the Pedersen commitment, and the logarithmic-round opening protocol — are inherited unchanged; only the carrier group and its scalars are fixed. Concretely, a polynomial of degree $< d$ over the Pallas scalar field is committed by a Pedersen vector commitment whose values lie in the Pallas point group.

Definition 5.6 (IPA scheme over Vesta). $\vdash$ Lean: `Kimchi.Commitment.IPA.ipaVesta`

The IPA commitment scheme over the Vesta curve is the generic scheme `ipa` of Definition 2.7 with the commitment group $\mathbb{G}$ taken to be the Vesta point group of Definition 5.3 and the scalar field $\mathbb{F}_r$ taken to be the Vesta scalar field of Definition 5.4. As for Pallas, every component of the generic scheme is inherited unchanged; the construction differs only in the choice of carrier curve. Because the Pasta curves form a cycle, the Pallas and Vesta instantiations are the two halves used together by the recursive proof system, but each is, on its own, an instance of the same generic IPA scheme.

5.3 Security at the Pasta curves

Theorem 5.7 (perfect correctness over Pallas). $\vdash$ Lean: `Kimchi.Commitment.IPA.ipaPallas_correctness`

The Pallas IPA scheme of Definition 5.5 satisfies perfect correctness (Definition 1.6): for every polynomial of degree $< d$ over the Pallas scalar field and every evaluation point, the honest commit-then-open interaction makes the verifier accept with probability 1.

Proof. The generic perfect-correctness theorem Theorem 3.1 holds for the scheme `ipa` over any prime-order commitment group with its scalar field. The Pallas instantiation of Definition 5.5 is exactly `ipa` with $\mathbb{G}$ the Pallas point group (Definition 5.1) and $\mathbb{F}_r$ the Pallas scalar field (Definition 5.2); both hypotheses of the generic theorem — that $\mathbb{G}$ is a commutative group acted on by the field $\mathbb{F}_r$ and that $\mathbb{F}_r$ is a field — are discharged by the `CompElliptic` instances for the Pasta curve. Specializing Theorem 3.1 at these choices yields the claim. $\square$

Theorem 5.8 (evaluation binding over Pallas). $\vdash \text{Lean: Kimchi.Commitment.IPA.ipaPallas_binding}$

The Pallas IPA scheme of Definition 5.5 satisfies evaluation binding (Definition 1.10): no efficient adversary can open a single Pallas commitment to two different evaluations at the same point, except with the negligible error of the generic bound.

Proof. The generic binding theorem Theorem 4.3 establishes evaluation binding for `ipa` over any prime-order commitment group with its scalar field, the binding error being controlled by the hardness of the discrete-logarithm relation among the SRS elements. The Pallas scheme is the instantiation of `ipa` at the Pallas point group (Definition 5.1) and Pallas scalar field (Definition 5.2), whose prime order is certified by `CompElliptic`; specializing Theorem 4.3 at this group and field gives the result. $\square$

Theorem 5.9 (perfect correctness over Vesta). $\vdash \text{Lean: Kimchi.Commitment.IPA.ipaVesta_correctness}$

The Vesta IPA scheme of Definition 5.6 satisfies perfect correctness (Definition 1.6).

Proof. Identical to the Pallas case (Theorem 5.7), specializing the generic theorem Theorem 3.1 at the Vesta point group (Definition 5.3) and Vesta scalar field (Definition 5.4). $\square$

Theorem 5.10 (evaluation binding over Vesta). $\vdash \text{Lean: Kimchi.Commitment.IPA.ipaVesta_binding}$

The Vesta IPA scheme of Definition 5.6 satisfies evaluation binding (Definition 1.10).

Proof. Identical to the Pallas case (Theorem 5.8), specializing the generic theorem Theorem 4.3 at the Vesta point group (Definition 5.3) and Vesta scalar field (Definition 5.4). $\square$